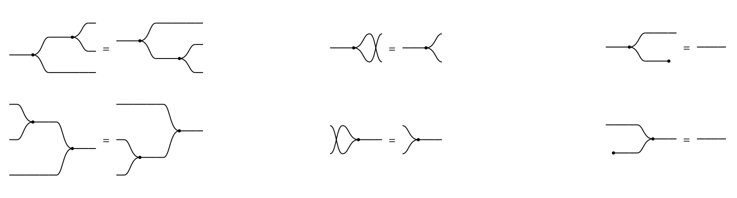
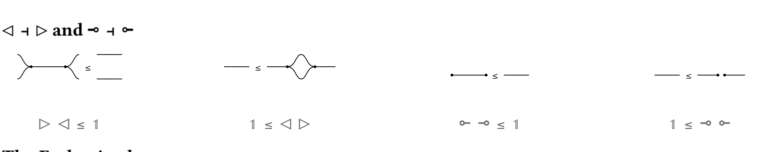
The allegory axioms, and what defines them in the Frobenius calculus

1. Cartesian bicategory of relations

$(◁,\ ⊸) ⊣ (▷,\ ⟜)$, commutative, **Frobenius**, arrows **lax**.

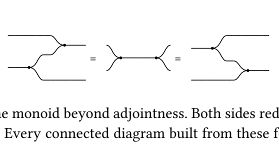


1.1. ◁ ⊣ ▷ and ⊸ ⊣ ⟜



$▷\ ◁ \le \mathbb{1}$ $\qquad \mathbb{1} \le ◁\ ▷$ $\qquad ⟜\ ⊸ \le \mathbb{1}$ $\qquad \mathbb{1} \le ⊸\ ⟜$

1.2. The Frobenius law



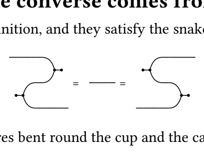
The only clause coupling the comonoid to the monoid beyond adjointness. Both sides reduce to the same picture, ▷ ◁ — merge, then copy — which is the two-in two-out **spider**. Every connected diagram built from these four generators collapses to the spider on its own inputs and outputs, and this equation is what starts that collapse. A **bubble** is the other order, ◁ ▷: copy then merge, a closed loop, which §4 uses.

1.3. Every arrow a lax comonoid homomorphism

$R\ ◁ \le ◁\ (R \otimes R)$ $\qquad R\ ⊸ \le ⊸$

2. Compact closed, and where the converse comes from

The cup ⟜ ◁ and the cap ▷ ⊸ come with the definition, and they satisfy the snakes:

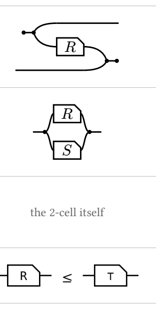


So every object is its own dual, $R°$ is R with its wires bent round the cup and the cap, and the four laws of ° are theorems rather than assumptions:

(i) $\mathbb{1}° = \mathbb{1}$ $\qquad$ (ii) $(R\ S)° = S°\ R°$ $\qquad$ (iii) $(R \otimes S)° = R° \otimes S°$ $\qquad$ (iv) $R \le S$ implies $R° \le S°$

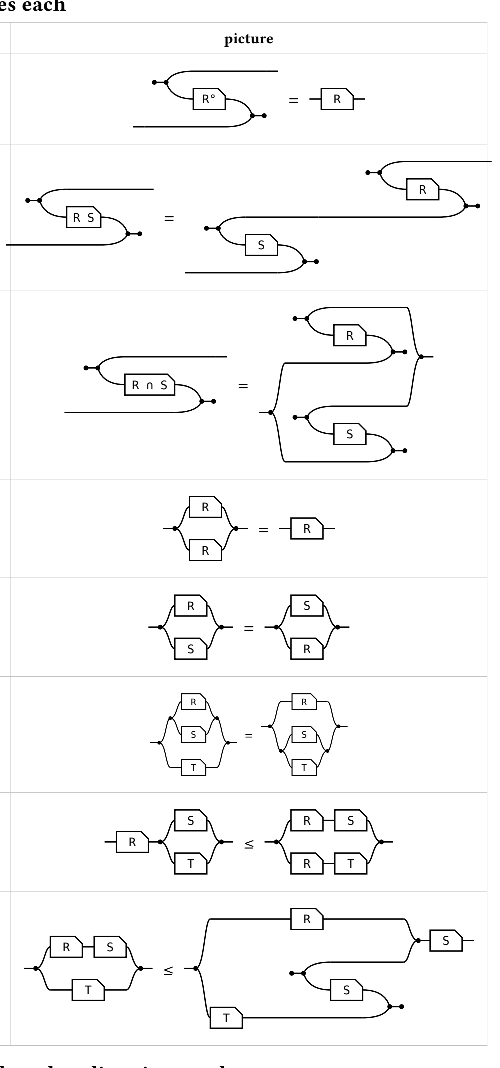
3. The allegory primitives are definitions here

primitive, and what defines it here	picture
reciprocation `R° := bend ((R ⊗ 𝟙) cap)`, not a generator.	
intersection `R ∩ S := ◁ (R ⊗ S) ▷` — copy, run both, merge.	
containment `R ⊑ S`, which an allegory **defines** as `R ∩ S = R`. Here ≤ is primitive and ∩ is derived, so the two directions are swapped.	the 2-cell itself
maximal morphism `⊤ := ⊸ ⟜`, free here; an allegory needs a unit object first.	

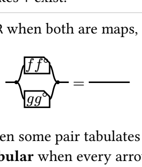


4. The eight axioms, and what proves each

axiom, and what supplies it	picture
`(R°)° = R` — the **snake**.	
`(R S)° = S° R°` — mirroring the picture.	
`(R ∩ S)° = R° ∩ S°` — the same mirroring; ◁ and ▷ are each other's mirror image.	
`R ∩ R = R` — the lax copy law, then `◁ ▷ = 𝟙` closes the bubble.	
`R ∩ S = S ∩ R` — cocommutativity and commutativity.	
`R ∩ (S ∩ T) = (R ∩ S) ∩ T` — coassociativity and associativity.	
`R (S ∩ T) ⊑ R S ∩ R T` — the lax copy law.	
`R S ∩ T ⊑ (R ∩ T S°) S`, the modular law — **adjoined** as an axiom by an allegory, which has no structure to derive it from. Here Frobenius and the lax copy law give it.	



5. Unitary and pre-tabular — what the other direction needs

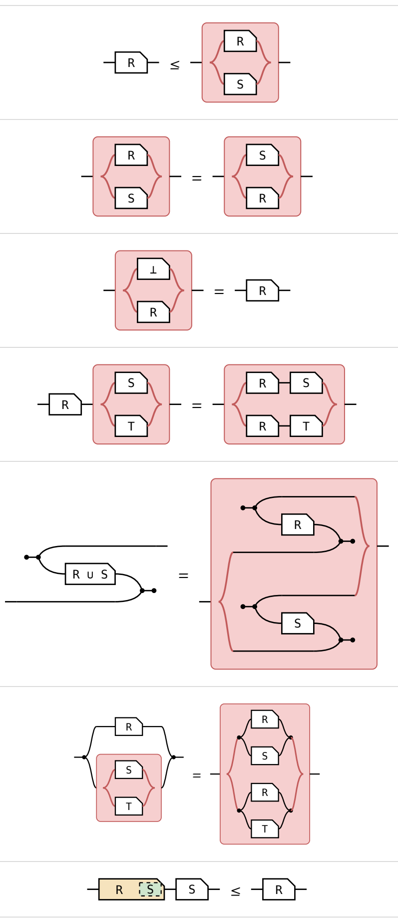
condition	allegory	Frobenius side
unitary	`⊤` is a **unit** when `𝟙 ⊤` is the largest endomorphism on it and every object carries an entire arrow to it. A unit is what makes `⊤` exist.	**free.** 𝕀 is the unit object, `𝟙 ≤ ⊸ ⟜` says ⊸ is entire, and `⊤ = ⊸ ⟜` is maximal.
pre-tabular	`f, g` **tabulate** `R` when both are maps, `R = f° g`, and  `R` is **tabular** when some pair tabulates it, the allegory **pre-tabular** when every arrow lies under a tabular one.	**this is what ⊗ is.** Given unitarity it reduces to tabulating each `⊤`, and a tabulation of `⊤` is an object `n ⊗ m` with its two projections. One side derives the product from tabulations, the other posits it.

Together: **cartesian bicategory of relations ≃ unitary pre-tabular allegory**.

6. Union and residual

Both are past what the definition can build: a union needs a **biproduct** on top of the Frobenius structure, a residual a **second composition** with linear adjoints. A union draws as a **tape** — the rounded wrapper is the second product, and its fork and join open and close a branch a particle takes exactly one of. A residual draws as **long division**, not as its own term, which is a composition against a complement.

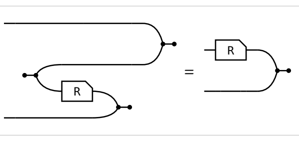
statement	picture
`R ⊑ R ∪ S` `union R S := ⟨R, S⟩ [𝟙, 𝟙]`: offer both branches, then forget which was taken.	
`R ∪ S = S ∪ R`	
`⊥ ∪ R = R` ⊥ routes through the zero object; there is no shape for it.	
`R (S ∪ T) = R S ∪ R T` — composition distributes over union, which ∩ does not.	
`(R ∪ S)° = R° ∪ S°`	
`R ∩ (S ∪ T) = (R ∩ S) ∪ (R ∩ T)` — what makes the whole **distributive**, and the one picture with both layers in it.	
`(R / S) S ⊑ R` — the residual is a lower bound of the arrows it is the greatest of.	



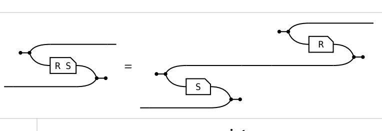
7. `(R S)° = S° R°`, drawn

Not a picture of a converse. An arrow is **the** converse of R as soon as it satisfies one equation between UNBENT arrows into 𝕀, and bending is a bijection, so the cup is never drawn: every row below has a cap and no cup, and "is a converse" shows up as a box on the **bottom** strand, mirrored.

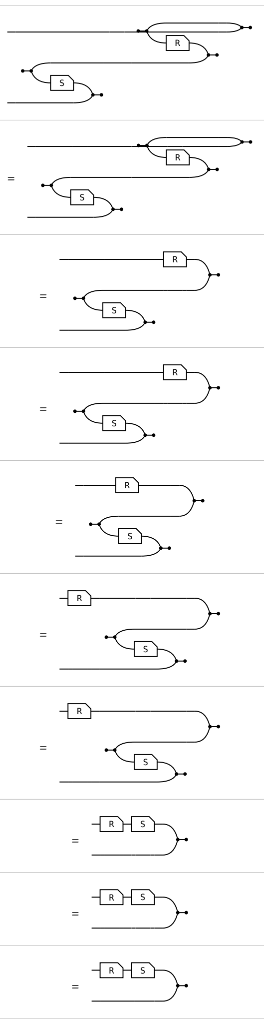
the slide — the one rule the proof uses, and it uses it twice



A box on the bottom strand facing a cap is that box on the top strand, upright.

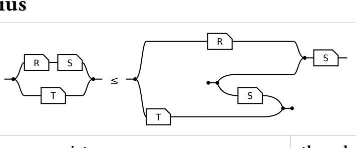
`(R S)° = S° R°` 

term	picture	the rule that reaches it
`(𝟙 ⊗ (S° R°)) cap`		**the start:** both boxes on the bottom strand, mirrored.
`= (𝟙 ⊗ S°) ((𝟙 ⊗ R°) cap)`		**⊗ is a functor.** Same picture.
`= (𝟙 ⊗ S°) ((R ⊗ 𝟙) cap)`		**the slide.** `R°` is the box nearest the cap: it leaves the bottom strand and comes back upright as R on the top.
`= ((𝟙 ⊗ S°) (R ⊗ 𝟙)) cap`		re-bracketing. Same picture.
`= (R ⊗ S°) cap`		**interchange.** In sequence and side by side are the same thing. A re-spacing only.
`= ((R ⊗ 𝟙) (𝟙 ⊗ S°)) cap`		**interchange back,** R first, so `S°` faces the cap.
`= (R ⊗ 𝟙) ((𝟙 ⊗ S°) cap)`		re-bracketing. Same picture.
`= (R ⊗ 𝟙) ((S ⊗ 𝟙) cap)`		**the slide,** on S. Both boxes now on top, upright, R then S.
`= ((R ⊗ 𝟙) (S ⊗ 𝟙)) cap`		re-bracketing. Same picture.
`= ((R S) ⊗ 𝟙) cap`		**interchange;** the two top boxes merge. This is `R S` unbent.

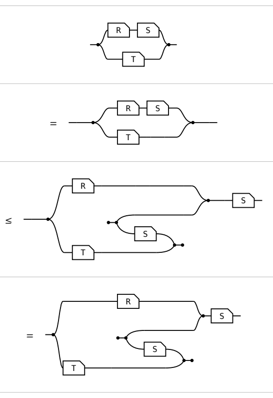


Seven of the nine steps change no picture: they are associativity, the unit laws and interchange, which a string diagram has in its geometry rather than as rewrites. The theorem is two slides.

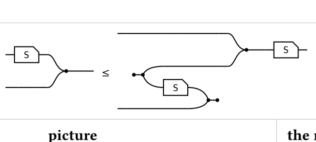
8. The modular law, from Frobenius

the modular law, `R S ∩ T ≤ (R ∩ T S°) S` 

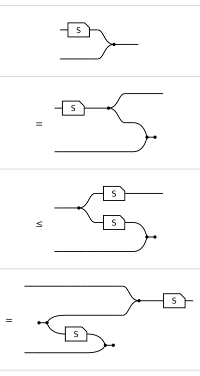
term	picture	the rule that reaches it
`(R S) ∩ T`		**the start:** `R S ∩ T`.
`= (◁ (R ⊗ T)) ((S ⊗ 𝟙) ▷)`		**Factor out the shared prefix:** `(R S) ⊗ T` is `(R ⊗ T) (S ⊗ 𝟙)`.
`≤ (◁ (R ⊗ T)) ((𝟙 ⊗ S°) (▷ S))`		**The merge slide.** S slides through the merge and reappears as `S°` on the other strand. The only inequality in the proof.
`= (R ∩ (T S°)) S`		**Fold the prefix back in;** the copy-merge pair reads as a meet again.



`S` occurs once on the left of the slide and twice on the right, and the lax copy law is the only law in the definition that may duplicate a box — so that is where the inequality has to be.

the merge slide, `(S ⊗ 𝟙) ▷ ≤ (𝟙 ⊗ S°) ▷ S` 

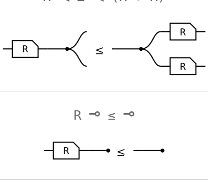
term	picture	the rule that reaches it
`(S ⊗ 𝟙) ▷`		**the start:** `(S ⊗ 𝟙) ▷`.
`= ((S ◁) ⊗ 𝟙) (𝟙 ⊗ cap)`		**Rebuild the merge** from a copy and a cap, the shape the lax copy law applies to.
`≤ ((◁ (S ⊗ S)) ⊗ 𝟙) (𝟙 ⊗ cap)`		**The lax copy law:** `S ◁` becomes `◁ (S ⊗ S)`, the box copied. The whole of the modular law, in one step.
`= (𝟙 ⊗ S°) (▷ S)`		**Reshape back,** and the surviving duplicate slides round the cap to become `S°`.

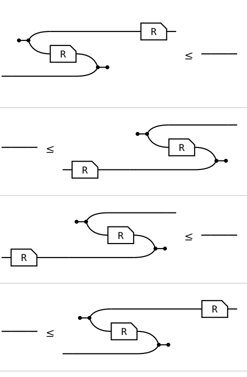


9. Maps

Every arrow is lax for ◁ and for ⊸; (ii) and (iv) above turn those into the same statements about ▷ and ⟜. All four hold for **every** arrow, their REVERSES do not, and each reverse holding is a property of the arrow. The right-hand column is the **adjoint** form, which is the definition used here because it is literally the allegory's `Simple` and `Entire`; that the two forms agree is a separate theorem, not proved here.

holds for every arrow	property	holds iff
`R ◁ ≤ ◁ (R ⊗ R)`	**(SV) single valued** `R° R ⊑ 𝟙`	
`R ⊸ ≤ ⊸`	**(TOT) total** `𝟙 ⊑ R R°`. With single valued, a map.	
`▷ R ≤ (R ⊗ R) ▷` — **not stated;** see below.	**(INJ) injective** `R R° ⊑ 𝟙`	
`⟜ R ≤ ⟜` — **not stated;** see below.	**(SUR) surjective** `𝟙 ⊑ R° R`, that is, `R°` entire.	

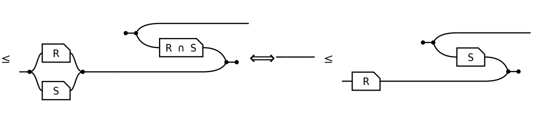




The two unstated ones are converse-images, and getting them needs `◁° = ▷` and `⊸° = ⟜` — a Frobenius computation at a COMPOSITE object, which is the clause the printed definition omits.

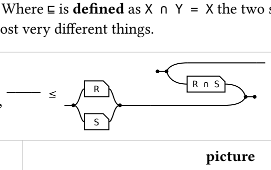
10. Entireness of an intersection

`R` is **entire** when `𝟙 ⊑ R R°`, which is (TOT) above. When is an **intersection** entire?

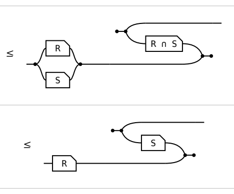


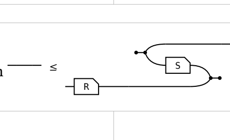
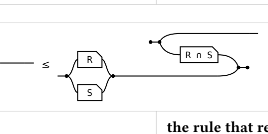
`Total (R ∩ S) ↔ 𝟙 ≤ R S°`

On the left `R ∩ S` is named **twice**; on the right R and S once each and the meet is gone. The meet is needed to **ask** whether something is entire, not to **answer** it. Where ⊑ is **defined** as `X ∩ Y = X` the two sides are one proposition; here ≤ is primitive, so both directions are real steps — and they cost very different things.

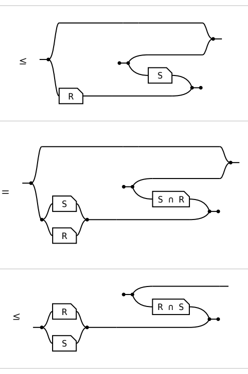
⟹ given `R ∩ S` entire,  show

term	picture	the rule that reaches it
`𝟙`		**the start:** `𝟙`.
`≤ (R ∩ S) (R ∩ S)°`		**the hypothesis:** `𝟙 ≤ P P°` at `P = R ∩ S`.
`≤ R S°`		**monotonicity, twice.** No modular law.



⟸ given  show `R ∩ S` entire, 

term	picture	the rule that reaches it
`𝟙`		**the start:** `𝟙`.
`≤ 𝟙 ∩ (R S°)`		**the hypothesis,** met with `𝟙`.
`= 𝟙 ∩ ((S ∩ R) (S ∩ R)°)`		`𝟙 ∩ R S° = 𝟙 ∩ (S ∩ R)(S ∩ R)°`. `R S°` relates `a` to `a′` when some `b` has `a R b` and `a′ S b`; the `𝟙 ∩` sets `a′ = a`, and it reads "some `b` has `a R b` and `a S b`" — one arrow of `R ∩ S`. Two arrows become one, and that is what needs the modular law.
`≤ (R ∩ S) (R ∩ S)°`		**`X ∩ Y ≤ Y`,** then `S ∩ R = R ∩ S`.



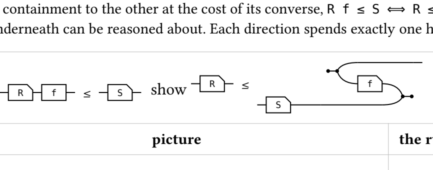
11. The gaps

- All eight axioms are theorems of the definition, the modular identity included.
- `▷◁≤𝟙` and `⟜⊸≤𝟙` are never used. They are there to make `◁ ⊣ ▷` and `⊸ ⊣ ⟜` genuine adjunctions, which is what pins `▷ = ◁°` and `⟜ = ⊸°`.
- Unproved: the merge and unit forms of the lax laws, and the agreement of the comonoid and adjoint forms of the map conditions. Both trace to the one omission above.
- The bridge runs one way only: a cartesian bicategory carries ⊗, an allegory carries nothing of the kind, which is what the two conditions supply.
- Everything here is a theorem OF the axioms. That relations satisfy them is not checked here.

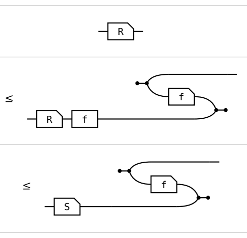
construct	counterpart	status
tabulation	none	`⊤ = ⊸ ⟜` is all the tabular content there is.
`∪, ⊥`	a **biproduct** ⊗ on top of the Frobenius structure	built, and drawn above. The calculus itself has no union.
`R / S`, complement	a **second composition** with linear adjoints	built, and drawn above, but axiomatised only: nothing instantiates the complement.
`Λ`, power transpose	none	open. No power object anywhere in this tower.
allegory ⟹ cartesian bicategory	rebuild ⊗ from tabulations	deliberately not built.

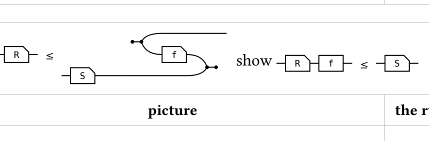
12. The shunting rule

A map moves from one side of a containment to the other at the cost of its converse, `R f ≤ S ⟺ R ≤ S f°`. It is how a function gets out of the way so the relation underneath can be reasoned about. Each direction spends exactly one half of **map** — never both.

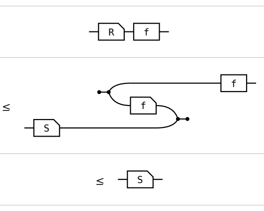
shunting right, ⟹ given … show … 

term	picture	the rule that reaches it
`R`		**the start:** R.
`≤ R (f f°)`		**total:** `𝟙 ≤ f f°`, so `f f°` may be inserted after R.
`≤ S f°`		Re-bracket to `(R f) f°` and apply the hypothesis.



shunting right, ⟸ given … show … 

term	picture	the rule that reaches it
`R f`		**the start:** `R f`.
`≤ S (f° f)`		Apply the hypothesis on the left; the two `f`s are now adjacent.
`≤ S`		**single valued:** `f° f ≤ 𝟙`, so the pair cancels.



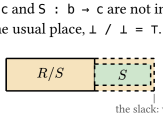
Shunting left is the mirror, same shape, map on the other side throughout. What the rule is, is an adjunction: `f ⊣ f°`, with **total** the unit and **single valued** the counit, so "`f` is a map" and "`f` has a right adjoint, namely `f°`" are one statement.

13. Division

Division is drawn and negation is not, because negation is not how programs get specified: the chapters where they are — greedy, thinning, dynamic programming, Horner, knapsack, paragraph formatting, string edit, bracketing, compression — use local completeness, division and power objects, and no Boolean structure at all. That also settles which picture of `/` is right. As a term a residual is a composition against a complement, which makes every homset Boolean; division asks for nothing of the kind, only the Galois connection `T ⊑ R/S ⟺ T S ⊑ R`, which `Rel(Set)` satisfies with a bare ∀, `(R / S) x y ≜ ∀z. S y z → R x z`.

Long division. `R / S` is how much of R you can lay down before S still fits in what is left. Read the bar as a composite: `R / S` first, then S, the two together inside R. Solid is what the equation pins — R's left edge, S's left edge, and the stretch before S. Dashed is the slack: nothing to the right of S is determined by `T S ⊑ R`, which is why the cancel law is `(R/S) S ⊑ R` and not an equality, and why the slack is a hairline — `R / S` is the **largest** such quotient.

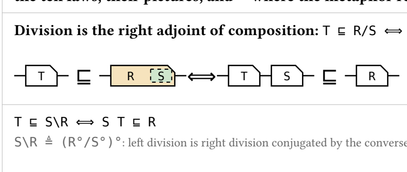
It is a **measurement**, not a containment: `R : a → c` and `S : b → c` are not in the same hom-set, so what sits inside R is the composite `(R/S) S`. Pushed too far the metaphor breaks in the usual place, `⊥ / ⊥ = ⊤`.



transcribed — the universal property has a statement, this metaphor does not

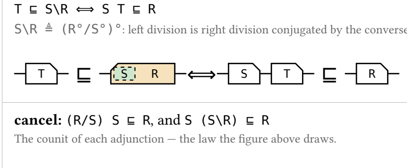
the ten laws, their pictures, and — where the metaphor reaches — the same law as long division

Division is the right adjoint of composition: `T ⊑ R/S ⟺ T S ⊑ R`



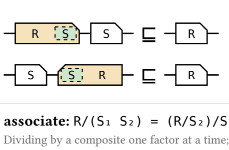
`T ⊑ S\R ⟺ S T ⊑ R`

`S\R ≜ (R°/S°)°`: left division is right division conjugated by the converse, not a second primitive.



cancel: `(R/S) S ⊑ R`, and `S (S\R) ⊑ R`

The counit of each adjunction — the law the figure above draws.

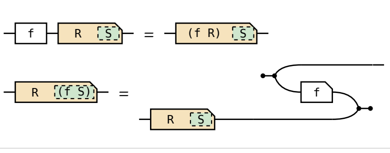


associate: `R/(S₁ S₂) = (R/S₂)/S₁`, and `(S T)\R = T\(S\R)`

Dividing by a composite one factor at a time; the bar shows what the label on the right hides — two tiles laid where there was one.

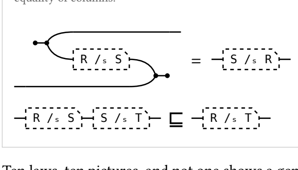
maps: `f (R/S) = (f R)/S`, and `R/(f S) = (R/S) f°`

The same three pieces bracketed two ways, which is the licence to write `f R / S`; then a map moves out of a denominator and reappears as `f°` outside the box. Both are the shunting rule, spent twice.



symmetric division: `(R/ₛS)° = S/ₛR`, and `(R/ₛS)(S/ₛT) ⊑ R/ₛT`

`R/ₛS ≜ (R/S) ∩ (S/R)°`. The meet is inside the definition, not the statement, so it stays under the label: `/ₛ` is converse-symmetric and transitive, like an equality of columns.



Ten laws, ten pictures, and not one shows a generator: ∩, ∪, ° and composition are what the Frobenius generators build, and `/` is none of those — it is posited, with nothing to unfold.